

CSE-214 OFFLINE - 3

Behavioural Design Patterns

Task 1: Disaster Alert Notification System

Topic: Observer Design Pattern | Marks: 15

Background

A government disaster management authority wants to develop a system named **BD Alert**. The system publishes emergency alerts for earthquakes, floods, and fires. Citizens may subscribe to one or more alert categories according to their needs. Whenever a new alert is published, every citizen subscribed to that category must be notified automatically. Citizens who are not subscribed to the category must not receive the notification.

System Requirements

The system must support the following functionality:

1. Register citizens in the system.
2. Subscribe a citizen to one or more disaster categories.
3. Update subscriptions or unsubscribe from a category at any time.
4. Publish an alert containing the title, category, affected location, severity level, and safety instructions.
5. Notify only the citizens subscribed to the relevant category.
6. Ensure that a newly subscribed citizen receives only future alerts.
7. Display the notifications received by each citizen.

Implementation & Demonstration Requirements

- Identify the behavioural design pattern that best represents the relationship between the alert categories and subscribed citizens.
- Use Java to implement the complete scenario.
- Demonstrate multiple citizens with different subscriptions.
- Publish at least one earthquake, one flood, and one fire alert.
- Show subscription updates and verify them by publishing another alert.

Task 2: BUET Final Result Publication System

Topic: Mediator Design Pattern | Marks: 10

Background

BUET follows a sequence of administrative steps before a student receives the final academic documents. The process involves the Department Office, the Office of the Controller of Examinations, the Directorate of Students' Welfare (DSW), and the student.

To avoid direct communication among all offices, the system will use a central result-processing coordinator. Every request, confirmation, and status update must pass through this coordinator. No office should directly call or control another office.

Required Processing Sequence

1. The Department Office confirms that the student has completed all academic requirements.
2. After receiving the departmental confirmation, the Controller of Examinations issues the office order for final result publication.
3. After the office order is issued, DSW issues the testimonial.
4. After all previous steps are completed, the Controller of Examinations issues the certificate and academic transcript.

System Requirements

The system must support the following functionality:

- Register the Department Office, Controller of Examinations, DSW, and students with the coordinator.
- Submit departmental confirmation for a student.
- Reject final-result publication when departmental confirmation is missing.
- Issue the final-result publication office order and notify the student.
- Allow DSW to issue the testimonial only after the office order has been issued.
- Issue the certificate and transcript only after all required steps are complete.
- Reject any request that violates the required sequence.
- Display the current processing status of a student.

Implementation & Demonstration Requirements

Identify the behavioural design pattern that best captures this centralized communication process and implement the complete scenario in Java. Your program must execute and demonstrate the following sequence:

1. An attempt to publish a result before departmental confirmation.
2. Submission of departmental confirmation.
3. An early attempt to issue the certificate or transcript.
4. Issuance of the final-result office order.
5. Issuance of the testimonial.
6. Issuance of the certificate and transcript.
7. Student notifications and the final status.